

Four Days to Submission

A hour-by-hour sprint playbook for a five-person team building a CCTV registry, unified viewer and ANPR watchlist platform on the Sentinel sandbox — from registration to submitted links.

SUBMISSIONS CLOSE Sun 7 Sept Upload + apply by this date	SHORTLIST 7 Sept, evening Same day as the deadline	ON-SITE EVENT 10–11 Sept i-Hub Gujarat • results 11th
---	---	--

WHAT WE ARE BUILDING Model 1 + Model 2. Model 1 (CCTV registry & GIS) is compulsory for every submission. Model 2 adds unified viewing and ANPR — which is what the live test case actually exercises.	WHAT WINS THE TEST CASE A registration number handed to you on the day, traced across the camera grid, with a timestamped route on a map — plus watchlist matching that fires alerts on its own.
THE ONE HARD DEPENDENCY Sandbox access. Live feeds sit behind login, and the government-feed demonstration is both mandatory and the first thing evaluated. Register today.	HOW TO USE THIS PLAYBOOK Everyone reads Part 0 and Part 2. Each person then works from their own pack in Part 3. The gates at the end of each day are team-wide — do not pass one by agreeing to fix it later.

Today, before any code

Three hours of work that decides whether the next four days are possible. Do these in order.

Nobody writes application code until step 5 passes.

BLOCKING

Without sandbox credentials there is no government-feed demonstration, and without that demonstration the submission fails evaluation area 1 regardless of how good everything else is. Registration is the critical path. Start it in the next ten minutes, not this evening.

Step 1 — Register (all five, now)

Go to sentinel.gujarat.gov.in/register and register under **Category 1 — Academic / Research / Startup**. Students, graduates, postgraduates and doctoral scholars all qualify; no DPIIT certificate is needed unless you are entering as a recognised startup.

- Check on the form whether one person registers the team or each member registers individually — do whichever it asks, and do it for everyone.
- Put the login in a shared note the whole team can reach. Do not let one person be the only one who can log in.
- If anything asks for a team name, agree it now. It appears on everything afterwards.

Step 2 — Get the sandbox details

Log in, open [the Resources page](#), and note the sandbox host. Read that page end to end — it is short, and it is the most useful technical document on the site.

Step 3 — Pull the camera catalogue

The catalogue is the contract. Camera ids change; URL patterns are not guaranteed. Everything downstream reads from here.

```
curl -s http://<host>/api/ingest > catalogue.json
python3 -m json.tool catalogue.json | head -60
```

Commit `catalogue.json` to the repository straight away. It is your camera inventory, your GIS layer and your onboarding demo, all from one call.

Step 4 — Prove a stream opens, two ways

```
# 1. RTSP for inference — TCP transport is mandatory
ffplay -rtsp_transport tcp rtsp://<host>:8554/stream/1
ffprobe -rtsp_transport tcp rtsp://<host>:8554/stream/1

# 2. Browser preview — paste into a tab
http://<host>:8889/stream/1/whep
http://<host>/live/stream/1/index.m3u8
```

If port 8554 is blocked on your network, fall back to the HLS endpoint and note it — you may need a different network on demo day.

GATE 0 — GO / NO-GO

Pass: catalogue downloaded, one RTSP stream decoding in ffmpeg, one stream playing in a browser. **Fail:** email sentinel.hackathon@gujarat.gov.in and call +91 95370 89982 (Mon–Sat, 10:00–18:00) the same hour. Do not spend a day debugging alone.

Step 5 — Team setup (30 minutes, together)

Repository	One GitHub repo, everyone with push access. Branch per person, merge daily. Make it public on Sunday, or add the evaluators.
Shared drive	One Google Drive folder for videos, PPT, HLD and the output report. Set sharing to Anyone with the link — Viewer now, so you are not fixing permissions on deadline day.
Comms	One group. Post the daily gate result in it every night.
Database	Everyone runs the same container: <code>postgis/postgis:16-3.4</code> . Agree the connection string tonight.
Python	3.11 or 3.12, one <code>requirements.txt</code> , everyone on the same versions.

Step 6 — Freeze the scope (30 minutes, everyone)

Read the in/out list in Part 5 aloud and agree it. After tonight nothing is added. Every hackathon team that fails does so by adding features on day three.

Who does what

Five roles, deliberately separated so nobody blocks anybody. Write the names in — ambiguity about ownership costs more than a missing skill.

ROLE	OWNS	DELIVERS BY SATURDAY NIGHT	NAME
A	Vision & ingest	RTSP capture that survives reconnects and loops; ANPR reading plates from live streams with track-level voting; the continuous indexing run	_____
B	Backend & data	Postgres/PostGIS schema; camera registry; detection index; watchlist matching; alerts over WebSocket; the REST API	_____
C	Frontend	GIS map with all cameras; multi-camera video wall; plate search; live alert panel; watchlist screen	_____
D	GIS, routing & reports	Camera locations plotted correctly; route reconstruction on the map; the output report of plates and timestamps	_____
E	Documents & submission	Solution PPT; HLD document; both demo videos; every submission link tested and working	_____

DO NOT CUT ROLE E

Five of the seven official evaluation areas are judged on documents and videos, not on running software. A full-time owner for the presentation, the HLD, the two demo videos and the submission mechanics is not overhead — it is where most of the marks are. Teams that leave this to Saturday night submit something incoherent and lose on completeness alone.

Rules of engagement

- **Daily gate at 21:00.** Fifteen minutes, standing up, everyone. State whether your gate item passes. If it does not, say so — the plan reshuffles around it the same night, not two days later.
- **Nobody works alone on a blocker for more than 90 minutes.** Post it in the group.
- **A works ahead of B and C.** If A's plate reads are late, B seeds the detections table with fake rows so C and D are never blocked. Agree the row shape on Thursday morning.
- **E interviews everyone daily.** Ten minutes each. E cannot write the HLD from guesses about what the system does.

The four days

Each day ends with one gate. The gate is a demonstration, not an opinion — someone shows the thing working on a screen.

DAY 0 Wednesday 3 September

Registered, feeds proven, scope frozen, environments running

All of Part 0. Nothing else. If Part 0 finishes early, each person installs their toolchain from their pack in Part 3 and runs its smoke test.

GATE 0

Catalogue on disk · one stream in ffplay · one stream in a browser · repo and database running on all five machines.

DAY 1 Thursday 4 September

Three tracks in parallel: pixels, data, screen

- A Capture wrapper: TCP transport, timestamps taken from PTS, reconnect with exponential backoff (2 s → 30 s), decoder warnings logged not fatal. Then get an off-the-shelf ANPR engine reading plates from saved frames, and point it at one live stream. **End of day: plate strings printing to console with PTS timestamps.**
- B Postgres + PostGIS up. Schema: `cameras`, `detections`, `watchlist`, `alerts`, `audit_log`. FastAPI skeleton. Load `catalogue.json` into `cameras`. **End of day: `GET /api/v1/cameras` returns GeoJSON.** Also seed 200 fake detection rows so C and D can build against real-shaped data.
- C React + Vite app. Leaflet map reading cameras from B's endpoint. Video wall: a 3×3 grid of `hls.js` players using the catalogue's HLS URLs. **End of day: map with every camera, and nine live tiles playing.**
- D Work through the catalogue's location fields. If they are names rather than coordinates, resolve them — this is the most likely unpleasant surprise of the week, so find it today. Build the department, codec and status layers. **End of day: cameras plotted in the right places with working layer toggles.**
- E Read the entire problem statement, all 50 FAQs and the Resources page. Become the person who knows the rules. Build the PPT and HLD skeletons from the official section lists (Part 4 of this playbook maps every required section to a source). **End of day: both skeletons exist, and every section that does not depend on the build is already written.**

GATE 1

Live video visible in a browser · plates being read from a real stream · camera list served from the database.

- A Scale to concurrent streams — start with ten workers. Add a tracker and **aggregate plate reads across each track**, emitting one best read per vehicle pass instead of forty noisy ones. Add Indian plate grammar correction. Write detections through B's API. **Then start the continuous run tonight and never stop it** (see the note below).
- B Watchlist table, CRUD and bulk import. Matching service: every detection is checked against the watchlist, exact then fuzzy, and a match writes an alert. WebSocket `/ws/alerts`. Search endpoint over plate (exact, partial, fuzzy via `pg_trgm`), time range and camera. Sightings-to-route endpoint.
- C Search screen with results and evidence crops. Live alert panel on the WebSocket. Watchlist management screen. **End of day: type a plate, see every sighting.**
- D Route reconstruction: order sightings by time, draw the polyline, compute speed between consecutive sightings, flag physically impossible transitions rather than hiding them. Build the output-report generator (plate, timestamp, camera, location → PDF and XLSX).
- E Write the PPT content: model choice with justification, architecture, analytics approach, watchlist correlation methodology, scalability. Write the HLD sections. Draft the shot list for the own-feed video. **End of day: PPT ~70%, HLD ~60%.**

THE SINGLE HIGHEST-LEVERAGE DECISION THIS WEEK

The grid is roughly twelve hours of footage per camera replayed on a loop. **Start the ANPR pipeline tonight and let it run continuously until submission.** By evaluation your index already holds every plate at every camera at every timestamp, so when the judges hand you a registration number the route renders instantly and completely, instead of you processing live in front of them. It is also, word for word, what the brief asks for: a solution that *continuously processes the CCTV feeds*.

GATE 2

A plate on the watchlist fires an alert in the UI within seconds · searching that plate returns its sightings · the route draws on the map.

MORNING — LAST BUILDING HOURS

- A Push the number of cameras under analytics as far as the hardware allows. **Measure and write down streams-per-machine, CPU and memory.** That number goes straight into the scalability section and is worth more than any claim.
- B Audit log on every search and export. Simple role model (state admin / department operator / viewer). Health endpoint showing per-camera ingest status. Then stop and harden.
- C Polish. This is the only day for it. Make the four screens that appear in the videos look finished; ignore the rest.
- D Generate the real output report from real government-feed data — detected plates with timestamps and camera locations. This is a required submission artefact, not a nice-to-have.
- E Set up recording, write the two scripts, rehearse both before hitting record.

AFTERNOON — RECORD BOTH VIDEOS

Video 1 — your own feed (2–3 minutes). Use your own footage: a phone video of a road with a readable plate is fine, restreamed as RTSP. Show, in this order — onboarding the feed, live detection with boxes and plate text, the detected plate matching a watchlist record, the alert appearing automatically, the sighting on the map.

Video 2 — the government feed. Show onboarding sandbox cameras from the catalogue, live viewing in the wall, ANPR output on those feeds, a plate search, and the reconstructed route across several cameras. Submit the output report alongside it.

EXPLICITLY DISQUALIFYING

The rules state that mock-ups, animations, simulated interfaces and concept videos without an operational backend will not be considered. Every frame of both videos must be real software doing real work. Do not screen-record a Figma file.

GATE 3

Both videos recorded and watched back by all five · PPT complete · HLD complete · output report generated · code frozen and tagged.

- Upload both videos to YouTube set to **Unlisted** — not Private, which the evaluators cannot open.
- Or upload to Drive/OneDrive with **Anyone with the link — Viewer**.
- **Open every link in an incognito window and check it plays.** A permissions error is the most common way a complete submission scores zero.
- Deploy the platform somewhere reachable and create a test login for the screening committee.
- Make the repository accessible and check the README explains how to run it.
- Final read of PPT and HLD by someone who did not write them.
- **Submit by 14:00.** Portal load on deadline day is real, and you want hours of margin, not minutes.

Shortlisting is announced the same evening. If you are through, the on-site round is 10–11 September at i-Hub Gujarat — rest on Monday, then prepare for a live demonstration on production feeds.

Resource pack, per person

Install list, the links that matter, the commands you will actually type, and which part of the Tech Doc to read before you start. Read only your own pack, plus the sandbox rules in Part 5.

INSTALL

```
pip install opencv-python av numpy fast-alpr supervision
# fallback OCR route if fast-alpr disappoints:
pip install ultralytics paddleocr paddlepaddle
sudo apt install ffmpeg
```

LINKS

fast-alpr — ONNX plate detection + OCR, works out of the box · github.com/ankandrew/fast-alpr

open-image-models — the plate detectors behind it · github.com/ankandrew/open-image-models

PaddleOCR — Apache-2.0 OCR if you build the two-stage pipeline yourself · github.com/PaddlePaddle/PaddleOCR

supervision — ByteTrack tracking and annotation, minimal setup · github.com/roboflow/supervision

Indian plate datasets, only if you fine-tune · Roboflow Universe, and github.com/sanchit2843/Indian_LPR

Sentinel Resources page — read it twice · sentinel.gujarat.gov.in/resource

THE CAPTURE WRAPPER — GET THIS RIGHT FIRST

```
import os
os.environ["OPENCV_FFMPEG_CAPTURE_OPTIONS"] = "rtsp_transport;tcp"
import cv2

cap = cv2.VideoCapture(url, cv2.CAP_FFMPEG)
ok, frame = cap.read()
pts_ms = cap.get(cv2.CAP_PROP_POS_MSEC) # timing comes from HERE, never time.time()
```

- **Never** use wall-clock arrival time. On connect the gateway replays its buffered group-of-pictures, so the first second arrives faster than real time and an arrival-timestamped tracker computes impossible velocities on every reconnect.
- **Ignore** `CAP_PROP_FPS`. It does not match the real delivery rate. Any speed or dwell figure derived from it will be wrong.
- Reconnect with exponential backoff, 2 s up to 30 s. Never in a tight loop.
- Decoder messages at join (`Error constructing the frame RPS` , `Could not find ref with POC`) are normal until the first IDR frame. Log them; do not abort.
- The footage loops. At the loop point the scene cuts hard. Reset trackers and any re-identification state rather than assuming continuity.
- Cameras differ in codec and resolution. No fixed-shape inference batch across all of them.

WHERE THE ACCURACY ACTUALLY COMES FROM

Two cheap steps, both frequently skipped, together worth far more than a better model. **One:** a vehicle is visible for 20–60 frames, so read the plate on every frame of the track and take a confidence-weighted per-character vote instead of trusting one frame. **Two:** Indian plates are `[2 letters][1–2 digits][1–3 letters][4 digits]`, so you know which positions must be alphabetic and which numeric — that makes $O \leftrightarrow 0$, $I \leftrightarrow 1$, $S \leftrightarrow 5$, $B \leftrightarrow 8$, $Z \leftrightarrow 2$ and $G \leftrightarrow 6$ deterministically correctable, and lets you validate the state code.

READ IN THE TECH DOC

§2.8 in full — the pipeline, the grammar post-processing, track-level voting, and the honest accuracy table you should quote from rather than overclaim.

DO NOT

Do not train a model. There is no time, and a pretrained engine plus the two steps above will beat a rushed training run.

INSTALL

```
docker run -d --name sentinel-db -p 5432:5432 \
  -e POSTGRES_PASSWORD=sentinel postgis/postgis:16-3.4
pip install fastapi uvicorn[standard] sqlalchemy geoalchemy2 \
  psycopg[binary] pydantic alembic websockets httpx
```

```
-- run once
CREATE EXTENSION postgis;
CREATE EXTENSION pg_trgm; -- this is what makes fuzzy plate search work
```

TABLES — AGREE THESE THURSDAY MORNING, EVERYONE BUILDS AGAINST THEM

cameras	id, native_id, name, department, location (geography Point), lat, lon, codec, resolution, rtsp_url, hls_url, whep_url, status, last_seen_at
detections	id, camera_id, plate_text, confidence, pts_ms, detected_at (UTC), track_id, crop_uri, vehicle_type, raw_reads (jsonb)
watchlist	id, entity_type (vehicle/person), plate_text, reason (stolen/wanted/missing/blacklisted), severity, case_ref, added_by, active
alerts	id, watchlist_id, detection_id, matched_at, match_type (exact/fuzzy), score, status (new/ack/resolved), acknowledged_by
audit_log	id, actor, action, object_type, object_id, purpose, case_ref, at

FUZZY PLATE SEARCH — THE QUERY THAT SAVES YOU ON DEMO DAY

```
SELECT *, similarity(plate_text, :q) AS score
FROM detections
WHERE plate_text % :q -- trigram match, tolerates one bad character
ORDER BY score DESC, detected_at;
```

ANPR will misread a character. Exact-match-only search will fail live in front of judges on a plate you did detect. Fuzzy search is not a nicety here.

LINKS

FastAPI · fastapi.tiangolo.com — its auto-generated OpenAPI page is also your API documentation deliverable

PostGIS · postgis.net/documentation · **pg_trgm** · postgresql.org/docs/current/pgtrgm.html

GeoAlchemy2 · geoalchemy-2.readthedocs.io

READ IN THE TECH DOC

Appendix A — the canonical camera schema. Do not copy all 90 fields; take the twenty you need and keep the shape. §1.7 and §2.7 list the endpoints worth exposing. §1.3 FR-6 covers the role model and audit design.

DO NOT

No Kafka, no Elasticsearch, no Keycloak. At fifty cameras Postgres does all of it, and the HLD describes the scale-out path instead. Those tools are a two-day detour you cannot afford.

INSTALL

```
npm create vite@latest sentinel-ui -- --template react-ts
npm i leaflet react-leaflet leaflet.markercluster hls.js
npm i -D tailwindcss @tailwindcss/vite
```

PLAYING THE FEEDS — NO RELAY NEEDED, THIS IS THE SHORTCUT

The sandbox already publishes HLS and WebRTC. You do not have to build a stream gateway, which is normally the hardest part of this kind of platform. A tile is roughly this:

```
import Hls from "hls.js";
const hls = new Hls({ lowLatencyMode: true });
hls.loadSource(`http://${host}/live/stream/${id}/index.m3u8`);
hls.attachMedia(videoEl);
```

Use HLS for the grid. Use the WHEP URL for one hero camera if you want to show sub-second latency in the video — it makes a good moment, but do not let it block the wall.

SCREENS TO BUILD, IN THIS ORDER

1. **Map** — all cameras, clustered, colour-coded by department, click a pin to open its live tile.
2. **Video wall** — 3×3 grid, drag a camera in, click a tile to maximise.
3. **Search** — plate input, results with evidence crop, timestamp and camera, and a "show route" button.
4. **Alerts** — live panel on the WebSocket, newest first, with acknowledge.
5. **Watchlist** — add and list entries. Plain table is fine.

Those five are exactly what appears in the demo videos. Anything else is out of scope.

LINKS

react-leaflet · react-leaflet.js.org — Leaflet is right at 50 cameras; ignore any advice about vector tiles, that was for 80,000

hls.js · github.com/video-dev/hls.js

markercluster · github.com/Leaflet/Leaflet.markercluster

READ IN THE TECH DOC

§1.3 FR-3 for the map layers worth having, §2.3 FR-4 for the operator interface. Both describe more than you should build — take the top third.

WATCH OUT

Nine simultaneous video elements leak memory if you do not tear players down when a tile changes camera or the tab is hidden. Call `hls.destroy()` on unmount. Test the wall running for twenty minutes before Saturday.

INSTALL

```
pip install geopandas shapely pyproj geopy requests
pip install openpyxl xlswriter weasyprint # output report as XLSX + PDF
```

FIRST JOB, THURSDAY MORNING — DE-RISK THE LOCATIONS

Open `catalogue.json` and find out what "location" actually contains. If it is coordinates, you are done in an hour. If it is place names, you need geocoding, and that is a half-day you must discover on Thursday rather than Saturday.

```
from geopy.geocoders import Nominatim
geo = Nominatim(user_agent="sentinel-hackathon")
geo.geocode("Lal Darwaja, Ahmedabad, Gujarat, India")
```

Rate-limit to one request per second, cache every result to a file, and hand-correct anything that lands in the wrong district. A camera in the Arabian Sea on demo day is an avoidable embarrassment.

ROUTE RECONSTRUCTION — THE MONEY SHOT

1. Take all sightings of the plate, sort by `detected_at`.
2. For each consecutive pair, compute great-circle distance and elapsed time, and from those the implied average speed.
3. Flag anything over ~150 km/h as a probable misread and show it as low-confidence — do not silently drop it. Showing the system catching its own errors reads as maturity.
4. Optionally snap the path to roads with the public OSRM demo server so the line follows streets instead of cutting across blocks. Small effort, large visual payoff.

```
https://router.project-osrm.org/route/v1/driving/
lon1,lat1;lon2,lat2?overview=full&geometries=geojson
```

THE OUTPUT REPORT — A REQUIRED ARTEFACT

The government-feed demonstration must be submitted "along with an output report showing detected vehicles or number plates with corresponding timestamps." Generate it from the database, not by hand: plate, confidence, camera id and name, department, location, UTC timestamp, evidence crop reference. Ship both XLSX and PDF.

LINKS

OSRM demo server · router.project-osrm.org · Nominatim · nominatim.org

Gujarat boundaries · github.com/datameet/maps · OSM extract · download.geofabrik.de/asia/india.html

ISRO Bhuvan — Indian-origin imagery, a small but real credibility signal in a government submission · bhuvan.nrsc.gov.in

READ IN THE TECH DOC

\$2.8.5 for route reconstruction, \$1.6 for the data-source table, \$1.8 if you have spare time and want to add a gap-analysis report as a bonus feature.

TOOLS

OBS Studio — screen recording, free · obsproject.com

Excalidraw or **draw.io** — architecture diagrams that do not look like clip art · excalidraw.com · app.diagrams.net

Google Slides / PowerPoint for the deck; export to PDF for submission

YOUR TWO DOCUMENTS, AND WHERE EVERY SECTION COMES FROM

This is the whole point of Part 4 — the Tech Doc already contains most of the architecture, sizing and security content the HLD asks for. Work through that mapping table rather than writing from scratch.

THE FIVE SUBMISSION ARTEFACTS

1. **Solution Presentation (PPT/PDF)** — model chosen with justification, overview, architecture, analytics approach, watchlist correlation method, technologies, scalability/security/interoperability, expected impact on policing.
2. **Technical Proposal / HLD** — architecture diagrams, heterogeneous integration approach, stream ingestion for dispersed sites, watchlist correlation and alerting, AI analytics, alert workflow, scalability to 80,000, and the technical information you would need from departments.
3. **Own-feed demo video** — 2–3 minutes, real working software.
4. **Government-feed demo video + output report** — onboarding, viewing, analytics output, plates with timestamps.
5. **Links** — unlisted YouTube or Drive/OneDrive with viewer access; optionally a hosted URL with test credentials and a Git repository.

RECORDING THAT DOES NOT LOOK AMATEUR

- Record at 1080p, 30 fps, and zoom the browser to ~125% so text is readable when compressed.
- Close every unrelated tab, notification and desktop icon.
- Write the script first, rehearse twice, then record. Voiceover afterwards over clean footage is easier than narrating live.
- Open on the result, not the login screen. The first ten seconds should show a plate being detected and an alert firing.
- Two to three minutes is a hard limit on video 1. Cut ruthlessly.

INTERVIEW THE TEAM DAILY

Ten minutes with each person, every evening. Ask what it does, what it cannot do, and what number they measured. The HLD is only credible if it describes the system that actually exists — and the honest limitations section is worth more marks than an overclaim that gets caught in questioning by NFSU and DA-IICT reviewers.

How to use the Technical Document

The companion Tech Doc was written against all four reference models and already covers most of what the HLD and the scalability section demand. This table maps each official requirement to the section that answers it. Person E works down this table; everyone else reads only the rows marked for their role.

OFFICIAL REQUIREMENT (FROM THE PORTAL)	TECH DOC SECTION	WHO USES IT
Proposed model with justification	§0.1 four-model comparison; §E.3 strongest combinations	E
Solution overview and objectives	§1.2 and §2.2 objectives and outcomes	E
High-level architecture diagrams	§1.9 and §2.9 architecture diagrams	E, B
Integrating heterogeneous cameras, NVRs and VMS	§2.3 FR-1 connectors; §2.7 protocol register; Appendix C.1–C.2	E, A
Ingesting streams from dispersed locations	§4.8.1 hierarchical edge → regional → core topology	E
Correlating feeds with watchlist databases	§2.3 FR-3.5 and §2.8.5; §3.3 FR-4 correlation rule design	E, B
AI analytics approach (ANPR, FRS, detection, tracking)	§2.8 complete; §4.8.3 for the additional analytics you are <i>not</i> building but should describe	E, A
Alert generation, prioritisation and workflow	§3.3 FR-5 workflow state machine and SLA design	E, C
Scalability to ~80,000 cameras	§4.9.1 bandwidth and storage arithmetic — the numbers are already worked out	E
Network and bandwidth planning	§4.9.1 (≈192 Gbps aggregate) and §4.8.1 (why edge-first cuts the WAN requirement by orders of magnitude)	E
Hot / warm / cold storage strategy	§4.3 FR-2 tiering table; §4.9.1 tiered volume model	E
AI processing capacity / GPU sizing	§4.9.2 sizing formula and the cost-versus-coverage argument	E
Disaster recovery strategy	§4.9.4 RPO/RTO table per data class and failure-mode design	E
Cybersecurity architecture	§4.3 FR-7 controls; §0.2.3 DPDP Act 2023 alignment	E
Cost-benefit analysis		E

OFFICIAL REQUIREMENT (FROM THE PORTAL)	TECH DOC SECTION	WHO USES IT
	\$4.9.2 tiered-analytics cost curve; \$4.10 infrastructure sizing	
Department-wise information requirements	Appendix A — the canonical camera schema <i>is</i> the answer: it is precisely the list of fields you need from every department to assess integration feasibility	E, B
Infrastructure sizing	\$1.10, \$2.10, \$4.10 prototype and production tables	E
Future roadmap	\$E.3 combinations; \$4.2 phased path from federation to consolidation	E
Integration with VAHAN / SARTHI / eGujCop / NAFIS	\$0.2.2 — these are closed systems; build contract-first adapters against documented mocks and say so	E, B

TWO ARGUMENTS THAT DIFFERENTIATE A SUBMISSION

Edge-first, argued with arithmetic. Eighty thousand cameras is roughly 192 Gbps and about 2 PB per day. Flat central ingestion is not viable, and showing the working — rather than a diagram — is the strongest competence signal available in the scalability section. It is done for you in \$4.9.1.

Closed government databases, handled correctly. VAHAN, SARTHI, eGujCop and NAFIS have no access route for a hackathon team. Do not pretend otherwise. Define the request/response contract, implement the adapter, mock the endpoint, and document exactly what changes when real credentials arrive. That is what "integration readiness" means, and stating it plainly beats a claim that collapses under one question.

Reference cards

Pin these somewhere everyone sees them.

Sandbox rules — eight things that otherwise cost a day

- 1 **Read the catalogue, never hardcode.** `GET /api/ingest` is the contract; camera ids and the camera set can change.
- 2 **Force RTSP over TCP** in every client. UDP fails across NAT and firewalls, and partial delivery produces corrupt frames that look exactly like model bugs.
- 3 **Drive all timing from PTS.** Buffered GOP replay on connect means arrival-time logic computes impossible velocities.
- 4 **Ignore the reported frame rate.** It does not match reality; anything time-derived from it is wrong.
- 5 **Reconnect with backoff**, 2 s to 30 s. Feeds are supervised and restart.
- 6 **Decoder warnings at join are normal** until the first IDR frame. Do not abort on them.
- 7 **The grid is not uniform.** Mixed H.264/H.265, mixed resolution and bitrate. Size buffers and batches per camera.
- 8 **Expect a scene discontinuity.** The footage loops with a hard cut; long-lived tracker and re-ID state must recover from it.

Two more: there is no file download — build against live capture from the start — and consume only, never publish to the gateway or call its control API. Each connected client gets its own copy of the stream, so open only the cameras you are actively processing.

The stack — deliberately small

BUILD THIS

- Python 3.12 + FastAPI
- PostgreSQL 16 + PostGIS + pg_trgm, in Docker
- React + Vite + TypeScript
- Leaflet for the map, hls.js for the wall
- Off-the-shelf ANPR + a tracker
- Local disk for evidence crops

DESCRIBE, DON'T BUILD

- Kafka, Elasticsearch, Kubernetes
- Keycloak — simple JWT roles are enough
- Face recognition
- Crowd counting, anomaly detection, vehicle re-ID
- Federation middleware (Model 3)
- Live VAHAN / eGujCop integration

The right-hand column belongs in the HLD as the scale-out path. Step 6 asks you to *present a strategy* for 80,000 cameras, not to demonstrate one — so describe it well and spend your four days on software that runs.

Scope freeze — the whole build, in one list

- ☐ Camera registry populated from the sandbox catalogue (Model 1, mandatory)
- ☐ GIS map with department, status and codec layers
- ☐ Multi-camera live viewing via HLS, with one WHEP hero tile
- ☐ ANPR on a continuous subset of cameras, with track-level voting and plate-grammar correction
- ☐ Detection index, searchable by exact, partial and fuzzy plate
- ☐ Watchlist database with bulk import
- ☐ Continuous matching and automatic real-time alerts

- ☐ Route reconstruction on the map with timestamped movement history
- ☐ Output report of plates and timestamps, as XLSX and PDF
- ☐ Audit log and a basic role model

Submission checklist & contingencies

Run the checklist on Sunday morning with two people, out loud. Evaluation area 7 is submission completeness, and it is the easiest one to lose for no reason.

Sunday morning checklist

- ☐ Solution Presentation exported to PDF, all required sections present
- ☐ HLD document exported to PDF, architecture diagrams legible at 100%
- ☐ Own-feed video, 2–3 minutes, shows working software end to end
- ☐ Government-feed video, shows onboarding + viewing + analytics on sandbox cameras
- ☐ Output report attached, with plates and timestamps from the government feed
- ☐ Videos uploaded as **Unlisted** on YouTube, or Drive/OneDrive set to Anyone with the link — Viewer
- ☐ **Every link opened and verified in an incognito window**
- ☐ Hosted platform URL live, with working test credentials written down in the submission
- ☐ Repository accessible, README explains how to run it
- ☐ All links pasted into the portal form and the form actually submitted
- ☐ Confirmation screenshot saved

If something goes wrong

PROBLEM	WHAT TO DO
Registration or verification stalls	Email sentinel.hackathon@gujarat.gov.in and call +91 95370 89982 the same hour. Helpdesk runs Mon–Sat, 10:00–18:00. Meanwhile build against your own RTSP feeds so the work is not idle.
Port 8554 blocked on your network	Use the HLS endpoint for viewing, and find a network where RTSP works for the inference pipeline. Test both from the venue network if you reach Phase 2.
Not enough compute for 50 streams	Run continuous analytics on 10–15 cameras and keep all 50 onboarded and viewable. Say so openly — a tiered analytics policy is a legitimate engineering decision and maps directly onto the statewide cost argument.
ANPR accuracy disappointing	Fuzzy search recovers most single-character errors. Report per-condition accuracy honestly with a failure gallery. An overclaim caught in questioning costs more than a modest number stated clearly.
A member drops out	C absorbs D's map work, E absorbs D's reporting. Protect A and E above all — without A there is no analytics, and without E there is no submission.
A feature is not finished on Saturday	Cut it and do not mention it. An incomplete feature shown in a video is worse than a smaller system shown working.

Contacts

Portal sentinel.gujarat.gov.in — registration, resources, live feeds, submission

Helpdesk sentinel.hackathon@gujarat.gov.in · +91 95370 89982 · Mon–Sat, 10:00–18:00

Venue i-Hub Gujarat, 10–11 September 2026

WHAT ACTUALLY DIFFERENTIATES YOU

Most teams will build a viewer and stop. The things that separate a shortlisted submission from the rest are all cheap: a route that renders instantly because you indexed continuously; a fuzzy search that recovers a misread plate live; an audit log and purpose-bound access that nobody else bothered with; measured throughput numbers instead of claimed ones; and a limitations section that names what does not work. Depth on a few things beats breadth across many.

Playbook prepared 3 September 2026 for a five-member Category 1 team. Companion reference: *Statewide CCTV Integration Programme — Technical Solution Document v1.0*. Figures for statewide scale are planning estimates and should be presented as such.